Pandas

📘 Chapter 1: Introduction to Pandas

📘 What is Pandas?

Pandas is a Python library used to work with data easily.
It helps you to store, view, change, and analyze data.

✅ Real-life Example:

Imagine you have a list of student names and their marks, like a table.
Just like Excel, you can manage this data in Python using Pandas.

📊 Example Table:

Name	Marks
Riya	90
Aryan	85

Using Pandas, you can:

- **Create this table in Python**
- **View and edit the data**
- **Find average marks**
- **Sort or filter the data**

📌 Key Points:

- **Pandas works like Excel in Python**
- **It makes data handling simple**
- **Useful for data analysis and data science**

📘 How to Install Pandas?

To use Pandas, you first need to install it in your Python environment.

✅ Steps to Install Pandas:

1. Open your command prompt or terminal.
2. Type the following command to install Pandas:

pip install pandas

3. Press Enter. This will download and install Pandas.

📌 What is pip?

- pip is the package installer for Python.
- It helps to install libraries (like Pandas) from the internet.

🔗 Verify Installation:

After installing, you can verify if Pandas is installed correctly by typing this in Python:

```
import pandas as pd
print(pd.__version__)
```

If you see the version number, Pandas is successfully installed!

📘 How to Import Pandas in Python?

After installing Pandas, you need to import it into your Python code to use it.

✅ Steps to Import Pandas:

1. In your Python script, type the following code:

```
import pandas as pd
```

2. Explanation:
   - import pandas loads the Pandas library.
   - as pd gives Pandas a shorter name (pd) so you can type less.

📌 Why use pd?

Instead of writing pandas every time, you can just use pd — it's quicker!

🔗 **Example:**

```
import pandas as pd


# Create a simple DataFrame
data = {'Name': ['John', 'Emma'], 'Age': [25, 28]}
df = pd.DataFrame(data)


print(df)
```

Output:

```
   Name  Age
0  John   25
1  Emma   28
```

📘 **Why is Pandas Important in Data Analysis and Data Science?**

Pandas is one of the most important tools for data analysis and data science because it makes working with data much easier and faster.

✅ **Key Reasons Why Pandas is Important:**

1. **Data Handling:**
   - **Pandas helps you organize and store data in a structured way (like in tables with rows and columns).**
   - **It can handle large datasets efficiently.**
2. **Easy Data Manipulation:**
   - **You can easily filter, sort, and group data with just a few lines of code.**
   - **Makes data cleaning and transformation simple.**
3. **Fast Data Analysis:**
   - **With Pandas, you can calculate mean, sum, median, and other statistical values quickly.**
   - **It helps in identifying trends, patterns, and outliers in data.**
4. **Support for Multiple Data Sources:**

- **Pandas can read data from different formats: CSV, Excel, SQL, and even JSON.**
- **This flexibility makes it great for working with real-world data.**

5. **Integration with Other Libraries:**
    - **Pandas works well with other Python libraries like NumPy, Matplotlib, and Scikit-learn.**
    - **It is an essential tool in the data science workflow, allowing smooth integration for analysis and visualization.**

📌 **Real-life Example in Data Science:**

Imagine you have data on customer purchases in a store. You can use Pandas to:

- **Clean the data (remove missing values).**
- **Group the data by customer type (new vs. returning customers).**
- **Calculate the average amount spent by each customer type.**

All of this can be done efficiently and quickly with Pandas!

Chapter 2: Series in Pandas

📘 What is a Pandas Series?

A **Series** in Pandas is like a **single column** of data, similar to a list or an array. It can hold any type of data (numbers, strings, etc.) and has **indexes** for each value, just like a row in a table.

✅ How to Create a Series:

1. **Using a list:**
   - You can create a Series from a **list** of values.

```
import pandas as pd


# Create a Series

data = [10, 20, 30, 40, 50]

series = pd.Series(data)


print(series)
```

Output:

```
0    10

1    20

2    30

3    40

4    50

dtype: int64
```

2. **Using a dictionary:**
   - You can also create a Series from a **dictionary** where keys are the indexes and values are the data.

```
import pandas as pd


# Create a Series from a dictionary

data = {'a': 10, 'b': 20, 'c': 30}

series = pd.Series(data)


print(series)
```

Output:

```
a    10
```

b 20

c 30

dtype: int64

📌 **Why Use Series?**

- **Simple data storage**: It helps store and access data easily, with labels (indexes) for each value.
- **Fast access**: You can quickly access and manipulate elements by their index.

📘 **How to Access Elements in a Pandas Series?**

You can access the elements of a Pandas Series using **indexes**, just like you access elements in a list or array.

✅ **Methods to Access Elements:**

1. **Access by Index (Position)**:
   - Use the index number to access elements in a Series (starting from **0**).

```
import pandas as pd

# Create a Series
data = [10, 20, 30, 40, 50]
series = pd.Series(data)

# Access the first element (index 0)
print(series[0])
```

Output:

10

2. **Access by Label (Index Name)**:
   - If your Series has custom index labels, you can access elements using those labels.

```
import pandas as pd

# Create a Series with custom indexes
data = {'a': 10, 'b': 20, 'c': 30}
```

```
series = pd.Series(data)


# Access element by label
print(series['a'])
```

Output:

```
10
```

3. **Access Multiple Elements:**
   - You can also access **multiple elements** by using a **list of indexes.**

```
import pandas as pd


# Create a Series
data = [10, 20, 30, 40, 50]
series = pd.Series(data)


# Access multiple elements
print(series[[0, 2, 4]])
```

Output:

```
0    10
2    30
4    50
dtype: int64
```

📌 **Summary:**

- You can access elements in a **Series** by their **position** (index number) or **label** (custom index).
- It's similar to how you access items in a list, but with **index labels.**

📘 Operations on Pandas Series

You can perform various operations on **Series** in Pandas, like **mathematical operations, comparisons, transformations,** and more.

✅ Common Operations on Series:

1. **Arithmetic Operations (Math operations):**
   - You can perform math operations like **addition, subtraction, multiplication,** etc., on Series elements.

```
import pandas as pd


# Create a Series

data = [10, 20, 30, 40, 50]

series = pd.Series(data)


# Add 10 to each element

result = series + 10

print(result)
```

Output:

```
0    20

1    30

2    40

3    50

4    60

dtype: int64
```

2. **Element-wise Operations:**
   - Operations are performed **element by element.** For example, dividing each value in the Series by 2.

```
# Divide each element by 2

result = series / 2

print(result)
```

Output:

```
go

CopyEdit

0    5.0

1    10.0
```

2 15.0

3 20.0

4 25.0

dtype: float64

3. **Comparison Operations:**
   - You can compare elements of the Series with **comparison operators** (like >, <, ==).

```
# Check which elements are greater than 30
result = series > 30
print(result)
```

Output:

0 False

1 False

2 False

3 True

4 True

dtype: bool

4. **Applying Functions:**
   - You can apply functions to each element in the Series using .apply().

```
# Apply a function to square each element
result = series.apply(lambda x: x ** 2)
print(result)
```

Output:

0 100

1 400

2 900

3 1600

4 2500

dtype: int64

5. **Summing Elements:**
   - You can quickly get the **sum** of all elements in a Series.

```
# Sum of all elements
result = series.sum()
```

```
print(result)
```

Output:

```
150
```

📌 **Summary:**

- **Arithmetic operations** can be applied element-wise (like adding a number to all elements).
- You can also **compare** elements and apply **functions** to modify the data.
- **Summing** and **other basic operations** like mean or median are also simple with Pandas.

📝 **Practice Questions:**

1. **Create a Series** from the list [10, 20, 30, 40, 50].
2. **Access the element** at index 2 in the Series [100, 200, 300, 400, 500].
3. **Add 5** to each element in the Series [1, 2, 3, 4, 5].
4. **Check which elements** in the Series [15, 25, 35, 45, 55] are greater than 30.
5. **Find the sum** of elements in the Series [10, 20, 30, 40].
6. **Apply the square** of each element in the Series [2, 4, 6, 8].

📂 Chapter 3: DataFrame in Pandas

Creating DataFrames in Pandas

A DataFrame is like a table that has rows and columns. It's used to store and manage data.

How to create a DataFrame:

1. **Using a Dictionary:**
   - **A dictionary has keys as column names and values as the data.**

```
import pandas as pd
data = {'Name': ['Alice', 'Bob', 'Charlie'], 'Age': [25, 30, 35], 'City': ['New York', 'Los Angeles', 'Chicago']}
df = pd.DataFrame(data)
print(df)
```

   - **This creates a table with columns: Name, Age, and City.**

2. **Using a List of Lists:**
   - **Here, each list represents a row of data.**

```
data = [['Alice', 25, 'New York'], ['Bob', 30, 'Los Angeles'], ['Charlie', 35, 'Chicago']]
df = pd.DataFrame(data, columns=['Name', 'Age', 'City'])
print(df)
```

3. **Reading from a CSV File:**
   - **You can also load a DataFrame from a CSV file.**

```
df = pd.read_csv('file.csv')
```

Accessing Rows and Columns in a DataFrame

Once you have a DataFrame, you will want to access specific data.

Accessing Columns:

1. **By Column Name:**
   - **To access a column, you can use its name:**

```
df['Age']  # This will show the 'Age' column
```

2. **Using Dot Notation:**
   - **If the column name does not have spaces, you can use this:**

```
df.Age  # This also shows the 'Age' column
```

Accessing Rows:

1. **Using loc[] (for index labels):**
   - **If you know the row index (label), you can access it like this:**

df.loc[1] # This will show the row with index 1 (second row)

2. **Using iloc[] (for position):**
    - **This is for accessing rows by position (like row number):**

df.iloc[1] # This will show the second row (position 1)

Accessing Specific Data (Row and Column):

- **You can access a specific value by combining row and column:**

CopyEdit

df.loc[1, 'City'] # This shows the value of 'City' in the second row

---

Quick Example:

```
import pandas as pd


# Creating a DataFrame
data = {'Name': ['Alice', 'Bob', 'Charlie'],
        'Age': [25, 30, 35],
        'City': ['New York', 'Los Angeles', 'Chicago']}
df = pd.DataFrame(data)


# Accessing the 'Age' column
print(df['Age'])


# Accessing the second row
print(df.loc[1])


# Accessing 'City' in the second row
print(df.loc[1, 'City'])


# Accessing 'Name' and 'City' columns
print(df[['Name', 'City']])
```

Key Points:

- **DataFrame is like a table with rows and columns.**

- You can create it using a dictionary or list of lists.
- To access a column, use df['ColumnName'].
- To access a row, use df.loc[RowIndex] or df.iloc[RowPosition].
- You can access a specific value using both row and column like df.loc[1, 'City'].

📘 DataFrame Attributes

Attributes are special variables that hold important information about the DataFrame. You can use them to get insights about the structure and data of your DataFrame.

🔷 Common DataFrame Attributes:

1. df.shape
   - This shows the number of rows and columns in the DataFrame.

```
df.shape  # Returns (number of rows, number of columns)
```

2. df.columns
   - This gives the names of the columns in the DataFrame.

```
df.columns  # Returns the column names
```

3. df.index
   - This shows the index labels of the rows.

```
df.index  # Returns the row index labels
```

4. df.dtypes
   - This returns the data types of each column (e.g., integer, string).

```
df.dtypes  # Returns the data types of each column
```

5. df.info()
   - This gives an overview of the DataFrame, including the number of non-null entries and data types.

```
df.info()  # Gives a summary of the DataFrame
```

6. df.head()
   - This shows the first 5 rows of the DataFrame by default. You can also specify how many rows to show.

```
df.head()  # Shows the first 5 rows

df.head(3)  # Shows the first 3 rows
```

7. df.tail()

- This shows the last 5 rows of the DataFrame by default. You can also specify how many rows to show.

```
df.tail()  # Shows the last 5 rows

df.tail(3)  # Shows the last 3 rows
```

📘 Basic Operations on DataFrames

You can perform many operations on DataFrames to manipulate and analyze data.

🔷 Basic Operations:

1. Adding a Column:
   - You can add a new column by assigning a value or a list to it.

```
df['Salary'] = [50000, 60000, 70000]  # Adds a new 'Salary' column
```

2. Removing a Column:
   - You can remove a column using drop() function.

```
df = df.drop('Salary', axis=1)  # Removes the 'Salary' column
```

3. Selecting Specific Columns:
   - You can select one or more columns using their names.

```
df[['Name', 'Age']]  # Selects 'Name' and 'Age' columns
```

4. Filtering Rows:
   - You can filter rows based on a condition.

```
df[df['Age'] > 30]  # Selects rows where 'Age' is greater than 30
```

5. Sorting the DataFrame:
   - You can sort the DataFrame by a specific column.

```
df.sort_values(by='Age', ascending=True)  # Sorts by 'Age' column in ascending order
```

6. Changing Values:
   - You can change specific values in the DataFrame.

```
df.loc[1, 'Age'] = 32  # Changes the 'Age' of row with index 1 to 32
```

7. Summary Statistics:
   - You can calculate the summary statistics (mean, median, etc.) of the DataFrame.

```
df.describe()  # Returns summary statistics for numerical columns
```

8. Checking for Missing Data:
   - You can check if any values are missing (NaN) in the DataFrame.

```
df.isnull()  # Returns a DataFrame with True for missing values

df.isnull().sum()  # Shows the total number of missing values in each column
```

9. **Renaming Columns:**
    - **You can rename columns using the rename() function.**

```
df = df.rename(columns={'Name': 'Full Name'})  # Renames 'Name' column to 'Full Name'
```

📘 **Example Code**

```
import pandas as pd

# Create a DataFrame
data = {'Name': ['Alice', 'Bob', 'Charlie'], 'Age': [25, 30, 35], 'City': ['New York', 'Los Angeles', 'Chicago']}
df = pd.DataFrame(data)

# Adding a new column 'Salary'
df['Salary'] = [50000, 60000, 70000]

# Removing the 'Salary' column
df = df.drop('Salary', axis=1)

# Selecting specific columns 'Name' and 'Age'
print(df[['Name', 'Age']])

# Filtering rows where Age > 30
print(df[df['Age'] > 30])

# Sorting by 'Age' column
print(df.sort_values(by='Age', ascending=True))

# Changing a specific value in the DataFrame
df.loc[1, 'Age'] = 32

# Summary statistics for the numerical columns
print(df.describe())
```

```
# Renaming a column
df = df.rename(columns={'Name': 'Full Name'})
print(df)
```

📌 **Key Points:**

- **Attributes like shape, columns, and index help you understand the structure of a DataFrame.**
- **Basic operations include adding/removing columns, filtering rows, sorting, and more.**
- **You can modify data in a DataFrame by changing values, renaming columns, or adding new ones.**

Practice Quations

1. Creating DataFrames

Create a DataFrame from a dictionary that contains the following data:

- **Name: ["Alice", "Bob", "Charlie"]**
- **Age: [25, 30, 35]**
- **Department: ["HR", "IT", "Finance"]**

After creating the DataFrame, print it.

---

2. Accessing Rows by Index

Using the DataFrame you created in the previous question, access and print the second row.

---

3. Accessing Columns by Name

Using the same DataFrame, access and print the "Age" column.

---

4. Accessing Specific Cell by Row and Column

Access and print the age of the second person in the DataFrame (using the row index and column name).

---

5. Adding a New Column

Add a new column named "Salary" to the DataFrame with values: [50000, 60000, 70000]. Print the updated DataFrame.

---

6. Basic Operations on DataFrames

- **Calculate the average age of all the people in the DataFrame.**
- **Add 1000 to each salary and print the updated DataFrame.**

📂 Chapter 4: Data Input and Output

📘 Reading CSV Files

You can load data from a CSV file into a DataFrame using Pandas.

How to Read CSV Files:

```
import pandas as pd


# Reading CSV file into a DataFrame

df = pd.read_csv('students.csv')


# Display the data

print(df)
```

- Example: Imagine a file named students.csv that has names and ages of students. After reading the file, you can see it as a table.

📘 Writing to CSV Files

Once you have a DataFrame, you can save it into a CSV file.

How to Write to CSV Files:

```
import pandas as pd


# Create DataFrame

data = {'Name': ['John', 'Anna', 'Steve'], 'Age': [20, 21, 22]}

df = pd.DataFrame(data)


# Write DataFrame to a CSV file

df.to_csv('output.csv', index=False)
```

- Example: After creating a table of names and ages, the data can be saved into a file named output.csv.

📘 Reading Excel Files

You can load data from an Excel file into a Pandas DataFrame.

How to Read Excel Files:

```
import pandas as pd
```

```
# Reading Excel file into a DataFrame

df = pd.read_excel('students.xlsx')


# Display the data

print(df)
```

- **Example: If you have an Excel file named students.xlsx with student data, this code loads the file into a table format.**

📘 Writing to Excel Files

Once you have data in a DataFrame, you can save it to an Excel file.

How to Write to Excel Files:

```
import pandas as pd


# Create DataFrame

data = {'Name': ['John', 'Anna', 'Steve'], 'Age': [20, 21, 22]}

df = pd.DataFrame(data)


# Write DataFrame to an Excel file

df.to_excel('output.xlsx', index=False)
```

- **Example: After creating a DataFrame with names and ages, the data can be saved into a file named output.xlsx.**

📘 Reading JSON Files

You can load data from a JSON file into a Pandas DataFrame.

How to Read JSON Files:

```
import pandas as pd


# Reading JSON file into a DataFrame

df = pd.read_json('data.json')


# Display the data

print(df)
```

- **Example: If you have a JSON file named data.json with data like:**

[{"Name": "John", "Age": 20}, {"Name": "Anna", "Age": 21}]

This code will load it into a DataFrame.

📘 Reading from SQL Database

You can load data directly from a SQL database into a Pandas DataFrame.

How to Read from SQL:

```
import sqlite3

import pandas as pd


# Connect to an SQLite database

conn = sqlite3.connect('students.db')


# SQL query to fetch data

query = "SELECT * FROM students"


# Reading from SQL into a DataFrame

df = pd.read_sql(query, conn)


# Display the data

print(df)


# Close the connection

conn.close()
```

- **Example: You can use this code to fetch data from an SQLite database named students.db and display it as a table.**

📘 Saving DataFrames to Files

Pandas allows you to save DataFrames in different formats.

How to Save DataFrames to Different Formats:

1. **Save as CSV:**

```
df.to_csv('students.csv', index=False)  # Saves DataFrame as CSV
```

2. **Save as Excel:**

```
df.to_excel('students.xlsx', index=False)  # Saves DataFrame as Excel
```

3. **Save as JSON:**

```
df.to_json('students.json', orient='records')  # Saves DataFrame as JSON
```

4. **Save to SQL:**

```
conn = sqlite3.connect('students.db')

df.to_sql('students', conn, if_exists='replace', index=False)

conn.close()  # Saves DataFrame to SQL
```

📌 **Key Points to Remember:**

- **CSV files are simple text files where data is separated by commas.**
- **Excel files are often used for more complex data and allow multiple sheets.**
- **JSON files store data in a structured format, commonly used for web data.**
- **SQL allows you to store data in databases and retrieve it for analysis.**

By using these commands, you can easily read and write data between Pandas DataFrames and various file formats like CSV, Excel, JSON, and SQL.

1. Reading a CSV File

Write a Python program to read a CSV file named "employees.csv" into a pandas DataFrame. Assume the file contains columns like "ID", "Name", "Age", and "Department". Print the first 5 rows of the DataFrame.

---

2. Writing to a CSV File

Using a DataFrame you create with the following data:

- **Name: ["Alice", "Bob", "Charlie"]**
- **Age: [25, 30, 35]**
- **Department: ["HR", "IT", "Finance"]**

Write this DataFrame to a CSV file named "employees_output.csv". Ensure the index is not saved in the file.

---

3. Reading Excel File

Write a Python program to read an Excel file named "sales_data.xlsx" into a DataFrame. Assume the Excel file contains multiple sheets, and you need to read data from a sheet named "2025". Print the first 5 rows of the data.

---

4. Reading JSON File

Write a Python program to read a JSON file named "student_data.json" into a pandas DataFrame. Print the structure of the DataFrame after loading the JSON file.

---

5. Reading SQL Database Table

Write a Python program to connect to a SQLite database named "company.db" and read the data from a table named "employees" into a DataFrame. Print the first 5 rows of the DataFrame.

---

6. Saving DataFrames to Files

Using the DataFrame from question #1, save the DataFrame as a JSON file named "employees.json" and then load it again into a new DataFrame. Print the loaded DataFrame.

🧹 Chapter 5: Data Cleaning

📘 Handling Missing Data

Missing data is common in real-world datasets. You can use Pandas to handle such missing data in various ways.

How to Handle Missing Data:

1. Checking for Missing Data:

You can check for missing data using isna() or isnull() functions.

```
import pandas as pd


# Sample DataFrame
data = {'Name': ['John', 'Anna', None, 'Steve'], 'Age': [20, None, 22, 23]}
df = pd.DataFrame(data)


# Check for missing data
print(df.isna())  # Returns a DataFrame with True for missing values
```

2. Removing Missing Data:

Use dropna() to remove rows with missing values.

```
df_clean = df.dropna()  # Removes rows with any missing data
print(df_clean)
```

3. Filling Missing Data:

Use fillna() to fill missing values with a specific value or a method.

```
df_filled = df.fillna({'Age': 0, 'Name': 'Unknown'})  # Replace missing values with specific values
print(df_filled)
```

📘 Removing Duplicates

Sometimes, your data may contain duplicate rows. Pandas makes it easy to remove them.

How to Remove Duplicates:

1. Removing Duplicate Rows:

Use drop_duplicates() to remove duplicate rows.

```
df = pd.DataFrame({'Name': ['John', 'Anna', 'John', 'Steve'], 'Age': [20, 21, 20, 23]})
df_clean = df.drop_duplicates()  # Removes duplicate rows
print(df_clean)
```

2. **Removing Duplicates Based on Specific Columns:**

You can also remove duplicates based on specific columns.

```
df_clean = df.drop_duplicates(subset=['Name'])  # Removes duplicates based only on 'Name' column
print(df_clean)
```

📘 Replacing Values

Sometimes, you need to replace specific values in your DataFrame.

How to Replace Values:

1. **Replacing Single Value:**

You can replace specific values using replace().

```
df = pd.DataFrame({'Name': ['John', 'Anna', 'Steve'], 'Age': [20, 21, 20]})
df['Age'] = df['Age'].replace(20, 22)  # Replace all 20 with 22 in 'Age' column
print(df)
```

2. **Replacing Multiple Values:**

You can replace multiple values in one go.

```
df = pd.DataFrame({'Name': ['John', 'Anna', 'Steve'], 'Age': [20, 21, 20]})
df = df.replace({'Age': {20: 22, 21: 23}})  # Replace 20 with 22 and 21 with 23
print(df)
```

📘 Renaming Columns

If the column names are not meaningful, you can rename columns to make them more understandable.

How to Rename Columns:

1. **Renaming Specific Columns:**

You can rename columns using the rename() method.

```
df = pd.DataFrame({'Name': ['John', 'Anna', 'Steve'], 'Age': [20, 21, 22]})
df = df.rename(columns={'Name': 'Full Name', 'Age': 'Age in Years'})  # Rename columns
print(df)
```

2. **Renaming All Columns:**

If you want to rename all columns at once, you can assign a list to the columns attribute.

```
df.columns = ['Full Name', 'Age in Years']  # Rename all columns
print(df)
```

📌 **Key Points to Remember:**

- **Handling Missing Data: You can handle missing data by removing it with dropna() or filling it with fillna().**
- **Removing Duplicates: Use drop_duplicates() to remove duplicate rows in your data.**
- **Replacing Values: Replace specific values using replace().**
- **Renaming Columns: Use rename() to rename columns, or assign new names directly to columns.**

📘 **Example Code:**

```
import pandas as pd


# Create a DataFrame with missing values

data = {'Name': ['John', 'Anna', None, 'Steve'], 'Age': [20, None, 22, 23]}

df = pd.DataFrame(data)


# Handling Missing Data: Fill missing values

df_filled = df.fillna({'Age': 0, 'Name': 'Unknown'})

print("After filling missing data:")

print(df_filled)


# Remove duplicates

df2 = pd.DataFrame({'Name': ['John', 'Anna', 'John', 'Steve'], 'Age': [20, 21, 20, 23]})

df_clean = df2.drop_duplicates()

print("\nAfter removing duplicates:")

print(df_clean)


# Replace values

df3 = pd.DataFrame({'Name': ['John', 'Anna', 'Steve'], 'Age': [20, 21, 20]})

df3['Age'] = df3['Age'].replace(20, 22)

print("\nAfter replacing values:")

print(df3)


# Rename columns
```

```
df4 = pd.DataFrame({'Name': ['John', 'Anna', 'Steve'], 'Age': [20, 21, 22]})

df4 = df4.rename(columns={'Name': 'Full Name', 'Age': 'Age in Years'})

print("\nAfter renaming columns:")

print(df4)
```

Practice quations

1. Handling Missing Data (NaN)

Create a DataFrame with the following data:

- **Name: ["Alice", "Bob", "Charlie"]**
- **Age: [25, None, 35]**
- **Department: ["HR", "IT", None]**

Handle the missing values by:

- **Replacing all missing values with "Unknown" for the string columns.**
- **Replacing the missing values in the "Age" column with the average age of the non-missing values.**

---

2. Removing Duplicates

Create a DataFrame with the following data:

- **Name: ["Alice", "Bob", "Alice", "Charlie"]**
- **Age: [25, 30, 25, 35]**
- **Department: ["HR", "IT", "HR", "Finance"]**

Remove any duplicate rows based on the "Name" column and print the resulting DataFrame.

---

3. Replacing Values

Create a DataFrame with the following data:

- **Name: ["Alice", "Bob", "Charlie"]**
- **Age: [25, 30, 35]**
- **Department: ["HR", "IT", "Finance"]**

Replace the "Finance" department name with "Accounting" and print the updated DataFrame.

---

4. Renaming Columns

Create a DataFrame with the following data:

- **ID: [1, 2, 3]**
- **Name: ["Alice", "Bob", "Charlie"]**
- **Age: [25, 30, 35]**

Rename the columns to "Employee_ID", "Full_Name", and "Age_Years". Print the DataFrame after renaming the columns.

---

5. Handling Missing Data by Dropping Rows

Create a DataFrame with the following data:

- **Name: ["Alice", "Bob", "Charlie"]**
- **Age: [25, None, 35]**
- **Department: ["HR", "IT", None]**

Drop any rows with missing values (NaN) and print the resulting DataFrame.

---

6. Replacing Specific Values in Multiple Columns

Create a DataFrame with the following data:

- **Name: ["Alice", "Bob", "Charlie"]**
- **Age: [25, 30, 35]**
- **Department: ["HR", "IT", "Finance"]**

Replace all instances of "IT" with "Information Technology" in the "Department" column and replace "Bob" with "Robert" in the "Name" column. Print the updated DataFrame.

🔍 Chapter 6: Data Selection and Filtering

📘 Using loc[] and iloc[]

In Pandas, you can select rows and columns of a DataFrame using loc[] and iloc[]. These are powerful tools for data selection.

1. Using loc[]:

- **loc[] is used to select rows and columns by labels (row and column names).**
- **It is label-based indexing.**

```
import pandas as pd


# Sample DataFrame
data = {'Name': ['John', 'Anna', 'Steve', 'Mike'], 'Age': [20, 21, 22, 23]}
df = pd.DataFrame(data)


# Selecting a row by label using loc[]
row = df.loc[1]  # Selects the row with index 1 (Anna's data)
print(row)


# Selecting a specific value
value = df.loc[1, 'Age']  # Selects the 'Age' of Anna
print(value)
```

2. Using iloc[]:

- **iloc[] is used to select rows and columns by integer position (index positions).**
- **It is integer-location based indexing.**

```
# Selecting a row by integer position using iloc[]
row = df.iloc[2]  # Selects the 3rd row (Steve's data)
print(row)


# Selecting a specific value by index
value = df.iloc[2, 1]  # Selects Steve's 'Age' using position
print(value)
```

📘 Conditional Filtering

Conditional filtering allows you to filter data based on specific conditions. This helps you extract data that matches certain criteria.

How to Filter Data Conditionally:

```
import pandas as pd


# Sample DataFrame
data = {'Name': ['John', 'Anna', 'Steve', 'Mike'], 'Age': [20, 21, 22, 23]}
df = pd.DataFrame(data)


# Filter data where Age is greater than 21
filtered_data = df[df['Age'] > 21]
print(filtered_data)
```

- **Example: The above code filters and returns rows where the Age is greater than 21.**

📘 **Filtering with Multiple Conditions**

You can also filter data using multiple conditions combined with & (AND) or | (OR) operators.

How to Apply Multiple Conditions:

1. **AND Condition (&):**

```
# Filter data where Age is greater than 20 AND Name is 'Anna'
filtered_data = df[(df['Age'] > 20) & (df['Name'] == 'Anna')]
print(filtered_data)
```

2. **OR Condition (|):**

```
# Filter data where Age is less than 22 OR Name is 'Steve'
filtered_data = df[(df['Age'] < 22) | (df['Name'] == 'Steve')]
print(filtered_data)
```

- **Note: Always remember to wrap each condition inside parentheses () while using & or |.**

📌 **Key Points to Remember:**

- **loc[]: Use for selecting rows and columns by labels (names).**
- **iloc[]: Use for selecting rows and columns by integer position (index).**
- **Conditional Filtering: Use boolean conditions to filter rows in a DataFrame.**
- **Multiple Conditions: Combine conditions using & (AND) or | (OR) for filtering based on multiple criteria.**

📘 **Example Code for Data Selection and Filtering:**

```
import pandas as pd

# Create a sample DataFrame
data = {'Name': ['John', 'Anna', 'Steve', 'Mike'], 'Age': [20, 21, 22, 23]}
df = pd.DataFrame(data)

# Using loc[] to select by label
row = df.loc[1]  # Select row with index 1 (Anna)
print("Row with loc[]:", row)

# Using iloc[] to select by position
row = df.iloc[2]  # Select 3rd row (Steve)
print("Row with iloc[]:", row)

# Conditional Filtering: Age greater than 21
filtered_data = df[df['Age'] > 21]
print("Data with Age > 21:")
print(filtered_data)

# Filtering with multiple conditions: Age > 20 AND Name is 'Anna'
filtered_data = df[(df['Age'] > 20) & (df['Name'] == 'Anna')]
print("Data with Age > 20 and Name is 'Anna':")
print(filtered_data)

# Filtering with multiple conditions: Age < 22 OR Name is 'Steve'
filtered_data = df[(df['Age'] < 22) | (df['Name'] == 'Steve')]
print("Data with Age < 22 or Name is 'Steve':")
print(filtered_data)
```

Practice Quations

1. Using loc[] to Select a Row

Create a DataFrame with these columns: "Name", "Age", "Department". Use loc[] to select the row where "Name" is "Bob".

---

2. Using iloc[] to Select a Row

Create a DataFrame with these columns: "Name", "Age", "Department". Use iloc[] to select the second row.

---

3. Conditional Filtering (Age > 30)

Create a DataFrame with columns: "Name", "Age", "Department". Filter and print rows where "Age" is greater than 30.

---

4. Conditional Filtering (Age > 25 and Department = IT)

Create a DataFrame. Filter rows where "Age" is greater than 25 and "Department" is "IT".

---

5. Using loc[] with Multiple Conditions

Create a DataFrame. Use loc[] to filter rows where "Age" is between 30 and 40 and "Department" is "IT".

---

6. Using iloc[] to Select Columns

Create a DataFrame. Use iloc[] to select the "Name" and "Age" columns from the first three rows.

These are simple questions focusing on using loc[], iloc[], and basic filtering conditions.

🔄 Chapter 7: Data Transformation

📘 Applying Functions: apply() and map()

1. apply()

The apply() function is used to apply a function to each column or row of a DataFrame.

- Use case: If you want to change every value in a column based on a function.

Example:

```
import pandas as pd


# Create a DataFrame
df = pd.DataFrame({'Age': [20, 21, 22]})


# Apply a function to add 1 to every value in the 'Age' column
df['Age'] = df['Age'].apply(lambda x: x + 1)


print(df)
```

Explanation: The lambda function adds 1 to each value in the 'Age' column.

2. map()

The map() function is used to apply a function element-wise to a Series (a single column).

- Use case: To map values in a column based on a dictionary or a function.

Example:

```
# Create a DataFrame
df = pd.DataFrame({'Name': ['John', 'Anna', 'Steve']})


# Map values in 'Name' column using a dictionary
df['Name'] = df['Name'].map({'John': 'Johnny', 'Anna': 'Annabelle'})


print(df)
```

Explanation: map() replaces 'John' with 'Johnny' and 'Anna' with 'Annabelle' based on the dictionary.

📘 Sorting and Ranking

1. Sorting

Sorting helps to arrange data in a specific order, either ascending or descending.

Example:

```
# Create a DataFrame
df = pd.DataFrame({'Name': ['John', 'Anna', 'Steve'], 'Age': [20, 21, 22]})


# Sort DataFrame by 'Age' column in descending order
df = df.sort_values(by='Age', ascending=False)


print(df)
```

Explanation: The data is sorted by the 'Age' column in descending order.

2. Ranking

Ranking assigns a rank to each value in a column.

Example:

```
# Add a 'Rank' column based on 'Age' values
df['Rank'] = df['Age'].rank()


print(df)
```

Explanation: The rank() function ranks the 'Age' column, where the highest value gets the rank 1.

📘 Changing Data Types

You can change the data type of a column using astype().

Example:

```
# Create a DataFrame
df = pd.DataFrame({'Age': [20, 21, 22]})


# Change the 'Age' column data type to string
df['Age'] = df['Age'].astype(str)


print(df)
```

Explanation: astype(str) converts the 'Age' column from integer to string.

📘 Working with Text Data

Pandas provides several string functions for text operations in DataFrames. These can be used on columns with string data.

1. Uppercase and Lowercase

Convert text to all uppercase or lowercase letters.

Example:

```
# Create a DataFrame
df = pd.DataFrame({'Name': ['John', 'Anna', 'Steve']})


# Convert the 'Name' column to uppercase
df['Name'] = df['Name'].str.upper()


print(df)
```

Explanation: The str.upper() function converts all names to uppercase.

2. Check if String Contains a Substring

You can check if a column contains a specific substring.

Example:

```
# Check if 'Name' contains 'John'
df['Contains John'] = df['Name'].str.contains('JOHN')


print(df)
```

Explanation: The str.contains() function checks if each name contains the substring 'JOHN'.

3. Replace Text

Replace a part of the string with new text.

Example:

```
# Replace 'John' with 'Jon' in the 'Name' column
df['Name'] = df['Name'].str.replace('JOHN', 'Jon')


print(df)
```

Explanation: The str.replace() function replaces 'JOHN' with 'Jon' in the 'Name' column.

4. Length of Text

You can find the length of each string in a column.

Example:

```
# Find the length of names in the 'Name' column
df['Name Length'] = df['Name'].str.len()
```

```
print(df)
```

Explanation: str.len() returns the number of characters in each name.

5. Splitting Strings

Split a string into parts based on a delimiter.

Example:

```
# Split the name into first and last names
df['First Name'] = df['Name'].str.split().str[0]
df['Last Name'] = df['Name'].str.split().str[1]


print(df)
```

Explanation: str.split() splits the name at the space, and .str[0] takes the first part (first name), and .str[1] takes the second part (last name).

📘 **Summary:**

- **apply(): Apply a function to a whole column/row.**
- **map(): Apply a function to each element in a column.**
- **Sorting: Order data by column in ascending/descending order.**
- **Ranking: Assign ranks to the values in a column.**
- **Changing Data Types: Use astype() to change column types.**
- **Text Functions: Manipulate text data using functions like str.upper(), str.contains(), str.replace(), etc.**

1. Using apply() to Apply a Function

Create a DataFrame with columns: "Name" and "Age". Use apply() to add 5 to each person's age and print the result.

---

2. Using map() to Apply a Function

Create a DataFrame with columns: "Name" and "Department". Use map() to convert each department name to uppercase and print the updated DataFrame.

---

3. Sorting Data

Create a DataFrame with columns: "Name", "Age", "Department". Sort the DataFrame by "Age" in ascending order and print the result.

---

4. Ranking Data

Create a DataFrame with columns: "Name" and "Salary". Rank the salaries in ascending order and print the DataFrame with the rank.

---

5. Changing Data Types

Create a DataFrame with columns: "Price" (as string) and "Quantity" (as integer). Convert "Price" to float and print the updated DataFrame.

---

6. Using String Functions on Text Data

Create a DataFrame with the column "Name". Use the str.lower() function to convert all names to lowercase and print the result.

Chapter -8

Data Manipulation

Topic: Adding/Removing Columns or Rows in pandas

🐼 **What is pandas?**

pandas is a Python library used for working with tables (called DataFrames). It helps to store, analyze, and manipulate data easily.

✅ **Adding Columns**

You can add a column to a DataFrame by assigning values to a new column name.

📌 **Syntax:**

```
DataFrame['new_column_name'] = values
```

✅ **Example:**

```
import pandas as pd


# Create a DataFrame

data = {'Name': ['Alice', 'Bob'], 'Age': [25, 30]}

df = pd.DataFrame(data)


# Add a new column 'City'

df['City'] = ['Delhi', 'Mumbai']


print(df)
```

📄 **Output:**

```
   Name  Age   City

0  Alice   25  Delhi

1    Bob   30  Mumbai
```

❌ **Removing Columns**

Use drop() function to remove a column.

📌 **Syntax:**

```
DataFrame.drop('column_name', axis=1, inplace=True)
```

- **axis=1 → means column**

- inplace=True → changes happen in the same DataFrame

❌ Example:

df.drop('Age', axis=1, inplace=True)

print(df)

📄 Output:

```
   Name   City
0  Alice  Delhi
1    Bob  Mumbai
```

✅ Adding Rows

You can add a new row using the loc[] method or append() (Note: append() is deprecated in newer pandas versions).

📌 Syntax using loc[]:

python

CopyEdit

```
DataFrame.loc[new_index] = [value1, value2, ...]
```

✅ Example:

python

CopyEdit

```
df.loc[2] = ['Charlie', 'Kolkata']
print(df)
```

📄 Output:

markdown

CopyEdit

```
     Name     City
0    Alice    Delhi
1      Bob   Mumbai
2  Charlie  Kolkata
```

❌ Removing Rows

Use drop() function to remove rows by index.

📌 Syntax:

python

CopyEdit

```
DataFrame.drop(index_number, inplace=True)
```

❌ Example:

python

CopyEdit

```
df.drop(1, inplace=True)
print(df)
```

📄 Output:

markdown

CopyEdit

```
   Name     City
0  Alice    Delhi
2  Charlie  Kolkata
```

✨ Summary Table

Action	Method Used	Key Parameter
Add Column	df['col'] = values	-
Remove Column	df.drop()	axis=1
Add Row	df.loc[]	-
Remove Row	df.drop()	index

Sorting and Duplicates in pandas

✅ 1. Sorting in pandas

🔷 There are 2 main functions:

- sort_values() → Sorts by column values
- sort_index() → Sorts by row index (numbers on the left)

✅ a) sort_values()

Used to sort the DataFrame by values in a column.

📌 Syntax:

```
df.sort_values('column_name', ascending=True)
```

🔷 ascending=True → sort in A to Z or small to big

🔷 ascending=False → sort in Z to A or big to small

✅ Example:

```
import pandas as pd


df = pd.DataFrame({
    'Name': ['Bob', 'Alice', 'Charlie'],
    'Age': [30, 25, 35]
})


# Sort by Age
sorted_df = df.sort_values('Age')
print(sorted_df)
```

📃 Output:

```
     Name  Age
1   Alice   25
0     Bob   30
2  Charlie   35
```

✅ b) sort_index()

Used to sort rows by index number.

📌 Syntax:

```
df.sort_index(ascending=True)
```

✅ Example:

```
df2 = df.sort_index(ascending=False)
print(df2)
```

✅ 2. Duplicates in pandas

🔷 Used to find or remove duplicate rows.

✅ a) duplicated()

Tells which rows are repeated.

📌 Syntax:

```
df.duplicated()
```

✅ Example:

```
df = pd.DataFrame({
    'Name': ['Ali', 'Bob', 'Ali'],
    'Age': [25, 30, 25]
})

print(df.duplicated())
```

📃 Output:

graphql

CopyEdit

```
0    False
1    False
2     True
```

Row 2 is a duplicate of row 0.

✅ b) drop_duplicates()

Removes duplicate rows.

📌 Syntax:

```
df.drop_duplicates()
```

✅ Example:

```
df2 = df.drop_duplicates()
print(df2)
```

📃 Output:

```
  Name  Age
0  Ali   25
```

1 Bob 30

✅ **Practice Questions**

🔷 **Q1. Create a DataFrame with columns Name and Marks. Sort the DataFrame by Marks in descending order.**

🔷 **Q2. You have a DataFrame with names in random order. Sort it alphabetically using sort_values().**

🔷 **Q3. Create a DataFrame and sort it using sort_index() in ascending order.**

🔷 **Q4. Create a DataFrame and sort it using sort_index() in descending order.**

🔷 **Q5. Check which rows are duplicates using duplicated() in a DataFrame with repeated names.**

🔷 **Q6. Write code to remove duplicate rows from a DataFrame using drop_duplicates().**

🔷 **Q7. Create a DataFrame with 5 rows. Sort the data by one column and then sort the same DataFrame by another column.**

🔷 **Q8. Create a DataFrame with columns Name, Age, and City. Sort it by two columns: first by City, then by Age.**

🔷 **Q9. How do you check for duplicate rows in a specific column (e.g., Name)?**

(Hint: Use duplicated(subset=['Name']))

🔷 **Q10. Create a DataFrame with duplicate rows. First, show the duplicates, then remove them, and print the final DataFrame.**

Chapter -9

Data Aggregation and Grouping in pandas

✅ 1. What is Data Aggregation?

👉 **Data aggregation means summarizing data.**
For example:

- **Find total marks**
- **Find average age**
- **Count how many students in each class**

✅ 2. What is groupby()?

👉 **groupby() groups data based on one column, and then you can apply aggregation functions on other columns.**

📌 **Syntax:**

```
df.groupby('column_name').function()
```

✅ **Example:**

```
import pandas as pd


data = {
    'Department': ['HR', 'HR', 'IT', 'IT', 'Finance'],
    'Salary': [30000, 35000, 40000, 42000, 50000]
}


df = pd.DataFrame(data)


# Group by Department and find total salary
result = df.groupby('Department').sum()
print(result)
```

📄 **Output:**

```
            Salary
```

Department

Finance 50000

HR 65000

IT 82000

✅ 3. Aggregation Functions

Here are some useful functions:

Function	Use
sum()	Total
mean()	Average
count()	Number of entries
max()	Maximum value
min()	Minimum value

✅ Example:

```
# Find average salary per department
df.groupby('Department').mean()
```

✅ Example with count:

```
# Count how many employees in each department
df.groupby('Department').count()
```

✅ 4. Multiple Aggregations with groupby()

You can apply multiple functions using agg().

📌 Syntax:

```
df.groupby('column').agg({'column2': ['sum', 'mean']})
```

✅ Example:

```
df.groupby('Department').agg({'Salary': ['sum', 'mean']})
```

✅ 5. What is a Pivot Table?

A pivot table reshapes data. It shows summary in a table format.

📌 **Syntax:**

```
pd.pivot_table(df, index='column1', values='column2', aggfunc='sum')
```

✅ **Example:**

```
data = {
    'Department': ['HR', 'HR', 'IT', 'IT', 'Finance'],
    'Employee': ['A', 'B', 'C', 'D', 'E'],
    'Salary': [30000, 35000, 40000, 42000, 50000]
}

df = pd.DataFrame(data)

pivot = pd.pivot_table(df, index='Department', values='Salary', aggfunc='sum')
print(pivot)
```

📄 **Output:**

```
            Salary
Department
Finance      50000
HR           65000
IT           82000
```

✨ **Summary Table**

Task	Code Example
Group data by column	df.groupby('col')
Total of each group	df.groupby('col').sum()
Average of each group	df.groupby('col').mean()
Count of each group	df.groupby('col').count()
Multiple aggregations	df.groupby('col').agg({'col2': ['sum', 'mean']})
Pivot table summary	pd.pivot_table(df, index='col1', values='col2')

Practice Questions: Data Aggregation and Grouping

🔷 **Q1. Create a DataFrame with columns Department and Salary. Use groupby() to find the total salary per department.**

🔷 **Q2. Use groupby() to find the average salary per department.**

🔷 **Q3. Count how many employees are there in each department using groupby() and count().**

🔷 **Q4. Use groupby() with agg() to get both sum and mean of salary per department.**

🔷 **Q5. Create a DataFrame with columns City, Department, and Salary. Group by City and show the total salary.**

🔷 **Q6. Create a pivot table to find total salary per department.**

🔷 **Q7. Create a pivot table to find average salary per department.**

🔷 **Q8. What is the difference between groupby() and pivot_table() in pandas?**

🔷 **Q9. Group a DataFrame by two columns (Department and City) and find the total salary for each group.**

🔷 **Q10. Create a DataFrame with Department, Gender, and Salary. Use pivot_table() to show average salary for each gender in each department.**